EncryptEdge Labs

Junior Cloud Security Engineer Internship Programme

Junior Cloud Security Engineer

Internship Task Report

Intern Full Name	Sukhmandeep Kaur
Intern Email	[redacted]
Cohort	Cohort 7, 2026
Task ID	CSE-AWS-L-003-04
Task Title	Secure AWS CLI Configuration and Identity-Based Access Operations
Skill Domain	AWS Cloud Security / Identity & Access Management (IAM)
Difficulty Level	Intermediate
CPE Credits	04 CPE Credits
Guiding Framework(s)	MITRE ATT&CK / NIST CSF / NIST SP 800-53 / CIS Benchmarks / ISO 27001 / OWASP / NICE Framework
Submission Date	22 july 2026

1.0 Task Overview

Task ID	CSE-AWS-L-003-04
Task Title	Secure AWS CLI Configuration and Identity-Based Access Operations
Skill Domain	AWS Cloud Security / Identity & Access Management (IAM)
Difficulty	Intermediate
Duration	*11* Hours
CPE Credits	4 CPE Credits
Cloud Platform(s)	*A*WS Free Tier
AWS Region(s) Used	us-east-2
Primary Tools	AWS CLI, AWS IAM, Boto3, jq, OpenAI ChatGPT
Target Environment	AWS Free Tier account, Cybr lab
MITRE ATT&CK IDs	T1078 (Valid Accounts), T1552.001 (Credentials in Files), T1087.004 (Cloud Account Discovery)
Guiding Frameworks	CIS AWS Foundations Benchmark v1.5, NIST SP 800-53, ISO 27001, OWASP Cloud Top 10
Regulatory Alignment	ISO 27001 A.9 Access Control, A.12 Operations Security; NICE PR-CDA-001; SFIA Level 3-4

1.1 Introduction

This task focused on strengthening AWS cloud security by managing IAM identities, access permissions, AWS CLI authentication, and temporary credentials protected by multi-factor authentication (MFA). The practical exercises demonstrated how different IAM users can be assigned different levels of access, how permissions are enforced through AWS APIs and the Boto3 SDK, and how temporary credentials can reduce the security risks associated with long-term access keys. These skills are important for cloud security engineers because improperly configured IAM permissions, exposed credentials, and weak authentication controls can lead to unauthorized access, privilege abuse, and cloud resource compromise.

1.2 Objectives & Learning Outcomes

Phase A — Understand the security implications of using the AWS CLI and how identity-based authentication mitigates risks.

Objective: Review how the AWS CLI interacts with IAM credentials and understand the identity context associated with an AWS CLI session.

Outcome: I used aws sts get-caller-identity to verify the current AWS identity and confirmed that the CLI session was authenticated as the IAM user test-user.

Objective: Differentiate between user-based and role-based authentication mechanisms.
Outcome: I examined the differences between IAM users and IAM roles, including the use of persistent credentials for users and temporary credentials commonly associated with assumed roles.

Objective: Understand the differences between static and temporary AWS credentials.
Outcome: I documented how static access keys are long-lived and require rotation, while temporary credentials are time-limited and include a session token.

Phase B — Configure multiple secure profiles in the AWS CLI using least privilege principles.

Objective: Create IAM users with different levels of access based on their intended responsibilities.
Outcome: I created the dev-user with restricted/read-only access and the ops-user with administrative access to demonstrate differentiated permissions.

Objective: Generate and configure AWS access keys for IAM users.
Outcome: I generated access keys for the IAM users and configured separate AWS CLI profiles using aws configure --profile.

Objective: Securely store AWS CLI credentials and understand appropriate credential protection.
Outcome: I verified that the AWS CLI profiles were stored in the AWS credentials configuration and reviewed the importance of restricting access to credential files.

Objective: Validate the identity associated with each AWS CLI profile.
Outcome: I used aws sts get-caller-identity --profile dev-user and aws sts get-caller-identity --profile ops-user to confirm that each CLI profile mapped to the correct IAM identity.

Phase C — Test least privilege enforcement by performing allowed and denied actions.

Objective: Test IAM permissions using AWS CLI commands and verify that access controls are enforced.
Outcome: I tested S3 and EC2 operations using the dev-user and ops-user profiles and observed successful and denied API operations based on the permissions assigned to each identity.

Objective: Demonstrate the principle of least privilege through restricted IAM access.
Outcome: The dev-user profile received authorization errors when attempting operations outside its permitted scope, demonstrating that IAM policies can restrict unauthorized actions.

Objective: Validate AWS authentication and authorization using the Boto3 SDK.
Outcome: I installed Python and Boto3 and executed a Python script using the dev-user profile to programmatically access Amazon S3, confirming that AWS IAM authorization is enforced through the SDK as well as the AWS CLI.

Objective: Document and analyze denied API responses as security evidence.
Outcome: I captured and documented AccessDenied responses to demonstrate the effectiveness of IAM authorization controls and identify restricted actions.

Phase D — Implement temporary session tokens using MFA for enhanced security.

Objective: Strengthen AWS account security by implementing multi-factor authentication (MFA).
Outcome: I enabled MFA as an additional authentication factor and documented how it reduces the risk of unauthorized access resulting from compromised passwords.

Objective: Generate temporary AWS session credentials using AWS Security Token Service (STS).
Outcome: I used the AWS STS get-session-token operation with MFA authentication to generate temporary credentials containing an access key, secret key, session token, and expiration time.

Objective: Configure and use temporary credentials through a dedicated AWS CLI profile.
Outcome: I configured the temporary credentials under the ops-temp AWS CLI profile and used the profile to authenticate AWS CLI requests.

Objective: Validate the expiration and functionality of temporary credentials.
Outcome: I verified the temporary session using aws sts get-caller-identity --profile ops-temp and documented the expiration time of the temporary credentials.

Phase E — Hosted Labs Integration

Objective: Reinforce AWS CLI and cloud security knowledge through practical external learning labs.
Outcome: I completed the required AWS CLI learning activities and used the practical exercises to reinforce the AWS CLI configuration, authentication, and security concepts covered in Phases A–D.

Objective: Apply AWS CLI skills through additional hands-on exercises and automation-focused learning.
Outcome: I completed the relevant AWS CLI training activities and documented the completion evidence through the required lab certificates.

Objective: Extend practical knowledge of AWS authentication and Boto3-based automation.
Outcome: I applied Boto3 concepts during the practical task and used Python to demonstrate programmatic interaction with AWS services; the optional PwnedLabs Boto3 activity can provide additional reinforcement where available.

Phase F — Final Security Assessment and ISO 27001 Alignment

Objective: Consolidate the results and evidence from all task phases into a professional security report.
Outcome: I organized the findings, command outputs, screenshots, and lab completion evidence into a structured report covering IAM, AWS CLI authentication, authorization, Boto3, MFA, and temporary credentials.

Objective: Identify cloud security threats associated with IAM and credential management.
Outcome: I developed a threat model addressing risks including credential leakage, excessive privileges, privilege escalation, compromised credentials, and misconfigured IAM policies.

Objective: Evaluate security controls that reduce the risk of unauthorized AWS access.
Outcome: I analyzed the effectiveness of least-privilege permissions, MFA, temporary credentials, secure credential storage, and IAM permission validation.

Objective: Align the security findings with relevant ISO/IEC 27001 control requirements and security principles.

Outcome: I mapped the practical security practices demonstrated in the task to relevant information security themes, including identity management, access control, authentication information, least privilege, and secure authentication.

1.3 Scope & Legal Authorisation

All activities conducted as part of this task were performed within authorised environments for educational and training purposes. The AWS CLI, IAM, STS, and Boto3 activities were conducted using an authorised personal AWS account and were limited to resources and identities created or accessed for this practical exercise. The hosted learning activities were completed within authorised training platforms and lab environments.

Authorised environments used:

- **Personal AWS Free Tier / authorised AWS account** — Used for AWS IAM user configuration, AWS CLI authentication, IAM permission testing, AWS STS temporary credentials, MFA configuration, and Boto3 testing. The AWS account was used only for authorised educational activities associated with this task.
- **Cybr training environment** — Used for authorised AWS CLI learning and hands-on training activities, including the *Getting Started with the AWS CLI* lab and other relevant AWS CLI exercises.
- **Local Windows environment** — Used to run the AWS CLI and Python/Boto3 scripts for the authorised AWS security exercises.

1.4 MITRE ATT&CK Cloud Threat Mapping

MITRE ATT&CK Technique ID	Technique Name	Relevance to This Task
T1078	**Valid Accounts**	The task involved authenticating to AWS using IAM user credentials and AWS CLI profiles. If legitimate credentials such as access keys are stolen or misused, an attacker could use them to access AWS resources while appearing to be an authorized user.
T1098	**Account Manipulation**	The task involved creating and configuring IAM users with different permission levels. In a real attack scenario, an adversary who gains sufficient privileges could create or modify IAM accounts, permissions, or access keys to maintain or expand access.
T1552.001	**Unsecured Credentials: Credentials In Files**	AWS CLI credentials are stored locally in AWS configuration and credential files. If these files are improperly protected or exposed, an attacker could obtain access keys and use them to access AWS resources.
T1078.004	**Valid Accounts: Cloud Accounts**	The AWS IAM users and profiles created during the task represent cloud identities that can be used to authenticate to AWS. Compromised cloud credentials could allow an attacker to operate using a legitimate AWS identity.
T1530	**Data from Cloud Storage Object**	The task included testing access to Amazon S3 through AWS CLI and Boto3. Although no unauthorized data extraction was performed, improperly configured S3 permissions could allow an attacker with valid credentials to access sensitive cloud storage data.
T1098.003	**Additional Cloud Roles**	AWS IAM roles and role-based authentication were reviewed as part of understanding user-based versus role-based authentication. An attacker with sufficient permissions could manipulate cloud roles or permissions to gain additional privileges.

2.0 Environment & Tool Setup

2.1 Cloud Architecture Overview

Cloud Resource / Component	Type / Service	Role in Task	Region / Config Notes
dev-user	AWS IAM User	Restricted user used to demonstrate least-privilege access and test denied permissions.	us-east-2
ops-user	AWS IAM User	Administrative user used to perform authorized AWS operations.	us-east-2
AWS CLI Profiles	AWS CLI	Used to authenticate and test AWS permissions using separate IAM identities.	dev-user and ops-user profiles
AWS STS	AWS Security Token Service	Used to generate temporary session credentials with an expiration time.	Temporary credentials used with ops-temp profile
MFA	AWS Multi-Factor Authentication	Used as an additional authentication factor for the temporary credential workflow.	Configured in the authorized AWS environment
Amazon S3	AWS Storage Service	Used to test IAM permissions through AWS CLI and Boto3.	S3 access tested using dev-user
Python / Boto3	AWS SDK for Python	Used to programmatically test S3 access using the dev-user profile.	Executed locally on Windows

2.2 AWS CLI & Tool Configuration

Tool / Service	Version	Installation Method	Purpose in This Task
AWS CLI	Version installed on Windows	Installed on Windows and configured using aws configure --profile	Used to authenticate to AWS, manage IAM profiles, test permissions, validate identities, and work with temporary credentials.
AWS IAM	AWS Managed Service	Configured through AWS Management Console	Used to create dev-user and ops-user and assign different permission levels.

Tool / Service	Version	Installation Method	Purpose in This Task
AWS STS	AWS Managed Service	No installation required; accessed through AWS CLI	Used to verify identities with get-caller-identity and generate temporary session credentials using get-session-token.
Python	Python 3.15	Installed locally on Windows	Used to run the Boto3 script for programmatic AWS service access.
Boto3	Installed using pip	Installed with python -m pip install boto3	Used to access Amazon S3 programmatically using the dev-user AWS profile and test IAM authorization.
AWS MFA	AWS Managed Security Feature	Configured through AWS Management Console and an authenticator app	Used as an additional authentication factor when working with temporary AWS credentials.

Key Configuration Commands Used

AWS CLI Profile Configuration

The AWS CLI was configured using separate profiles for the restricted dev-user and administrative ops-user IAM identities.

aws configure --profile dev-user

aws configure --profile ops-user

The profiles were configured with the appropriate access keys, secret access keys, and the AWS region:

Region: us-east-2

Output format: json

Verify AWS CLI Profile Configuration

The configuration of the ops-user profile was verified using:

aws configure list --profile ops-user

The output confirmed that the ops-user profile was configured and that the credentials were being loaded from the AWS shared credentials file.

Verify AWS Identity

The AWS identity associated with the initial CLI session was verified using:

aws sts get-caller-identity

The command successfully returned the AWS account, user ID, and ARN, confirming that the AWS CLI was authenticated.

The identity output confirmed the IAM user:

arn:aws:iam::*********:user/test-user

Verify Individual IAM Profiles

The identities associated with the configured profiles were verified using:

aws sts get-caller-identity --profile dev-user

aws sts get-caller-identity --profile ops-user

These commands were used to confirm that the AWS CLI profiles were correctly associated with their intended IAM users.

Python and Boto3 Configuration

Boto3 was installed locally using:

python -m pip install boto3

The Python script was executed using:

python C:\Users\Dell\Documents\Encrypt-Labs\Week3\PhaseC\phase_c.py

2.3 AI Tool Usage Declaration

AI Tool	Used For	What Was NOT Done with AI
ChatGPT	Used to explain AWS IAM, AWS CLI, STS, MFA, temporary credentials, Boto3, and cloud security concepts; provide guidance for troubleshooting AWS CLI and Python/Boto3 errors; assist with organizing and drafting report sections; and explain the meaning of task instructions and evidence requirements.	ChatGPT did not execute AWS commands, access the AWS account, configure AWS resources, generate or capture screenshots, create technical evidence, or perform the practical lab activities. All AWS CLI commands, configurations, IAM changes, testing, and screenshots were performed by the student.

All technical findings, AWS CLI outputs, configurations, screenshots, and evidence presented in this report are the student's own work. AI assistance was used only for explanation, troubleshooting guidance, organization, and documentation support.

3.0 Methodology — Phase Execution

.**Phase A:** Understand the security implications of using the AWS CLI and how identity-based authentication mitigates risks

Phase Objective	Understand the security implications of using the AWS CLI and how identity-based authentication mitigates risks.
Estimated Time	60 minutes
Evidence File(s)	PhaseA_identity.png, Phase_A_AWS_IAM_Credentials_and_Identity_Context.pdf

Actions Taken

During Phase A, I reviewed the relationship between the AWS Command Line Interface (AWS CLI) and AWS Identity and Access Management (IAM) credentials. I examined how the AWS CLI uses configured credentials and profiles to authenticate requests to AWS services. I also reviewed the differences between IAM users and IAM roles, focusing on how users generally authenticate using long-term credentials while roles are commonly associated with temporary security credentials.

To verify the identity associated with the active AWS CLI session, I executed the following command:

aws sts get-caller-identity

The command successfully returned the AWS account information, user ID, and Amazon Resource Name (ARN). The output confirmed that the active identity was the IAM user test-user.

{

"UserId": "*************",

"Account": "**********",

"Arn": "arn:aws:iam::************:user/test-user"

}

This confirmed that the AWS CLI was successfully authenticated and that the current identity context could be verified through AWS Security Token Service (STS).

I also reviewed the security differences between static and temporary credentials. Static access keys generally remain valid until they are manually rotated or revoked, which creates a greater risk if they are exposed or stolen. Temporary credentials have a limited validity period and include a session token, reducing the potential impact of credential compromise. MFA can provide an additional layer of protection when obtaining temporary credentials.

Key Observations

- Confirmed that AWS CLI authentication is linked to IAM credentials and profiles.
- Verified the active AWS identity using aws sts get-caller-identity.
- Learned that temporary credentials are safer than long-term static keys because they expire automatically.
- Identified credential leakage and excessive permissions as key cloud security risks.

Evidence Produced

phaseA_identity.png — Screenshot of the aws sts get-caller-identity command output confirming the authenticated AWS IAM identity.

Phase_A_AWS_IAM_Credentials_and_Identity_Context.pdf — Notes documenting AWS CLI authentication, IAM users versus roles, and differences between static and temporary credentials.

Phase B: Configure multiple secure profiles in the AWS CLI using least privilege principles.

Phase Objective	Configure multiple secure profiles in the AWS CLI using least privilege principles.

Estimated Time	90 minutes
Evidence File(s)	PhaseB-Dev-User.png, PhaseB-ops-user.png

Actions Taken

In Phase B, I configured separate IAM users and AWS CLI profiles to demonstrate different levels of access.

First, I created the dev-user IAM user with restricted/read-only permissions and the ops-user IAM user with administrative permissions. Access keys were generated for both users through the AWS Management Console.

I then configured separate AWS CLI profiles using:

aws configure --profile dev-user

The profile was configured with the dev-user access key, secret access key, AWS Region us-east-2, and JSON output format.

The administrative profile was configured using:

aws configure --profile ops-user

The ops-user access key, secret access key, AWS Region, and output format were entered when prompted.

I verified the ops-user profile configuration using:

aws configure list --profile ops-user

The relevant output confirmed that the profile was correctly configured:

NAME : VALUE : TYPE

profile : ops-user : manual

access_key : **********4ITY : shared-credentials-file

secret_key : **********JoU2 : shared-credentials-file

region : us-east-2 : config-file

The AWS CLI profiles were then validated by checking the identity associated with each profile:

aws sts get-caller-identity --profile dev-user

aws sts get-caller-identity --profile ops-user

The commands returned the AWS account information and ARN associated with the respective IAM users, confirming that the profiles were successfully linked to their intended identities.

Key Observations

- Different IAM users can be assigned different levels of access based on their responsibilities.
- Separate AWS CLI profiles allow users to switch between IAM identities without changing the main configuration.
- Least-privilege permissions help reduce the impact of unauthorized access.
- Access keys must be securely stored and protected from exposure.

PhaseB-Dev-User.png — Screenshot showing AWS CLI profiles configured for dev-user.

PhaseB-ops-user.png — Screenshot showing AWS CLI profiles configured for ops-user.

Phase C: Test least privilege enforcement by performing allowed and denied actions.

Phase Objective	Test least privilege enforcement by performing allowed and denied actions.
Estimated Time	120 minutes
Evidence File(s)	Bot_dev-user.png, phase_c.py, PhaseC-EC2-Dev-user.png, PhaseC-EC2-Ops-user-Permitted.png, PhaseC-S3-Denial-Dev-User.png, PhaseC-S3-ops-user.png

Actions Taken

In Phase C, I tested the permissions assigned to the dev-user and ops-user IAM identities using the AWS CLI. The purpose was to confirm that the restricted dev-user could perform permitted actions but would be denied access to operations outside its assigned permissions.

I first attempted to list IAM users using the restricted dev-user profile:

aws iam list-users --profile dev-user

The command returned an AccessDenied error:

An error occurred (AccessDenied) when calling the ListUsers operation:

User: arn:aws:iam::**********user/dev-user is not authorized to perform:

iam:ListUsers

because no identity-based policy allows the iam:ListUsers action

This demonstrated that the dev-user did not have permission to perform the iam:ListUsers action.

I then used Python and the Boto3 SDK to test programmatic access to Amazon S3. The Python script created a Boto3 session using the dev-user AWS CLI profile:

import boto3

session = boto3.Session(profile_name='dev-user')

s3 = session.client('s3')

response = s3.list_buckets()

print(response)

The script was executed locally using:

python C:\Users\Dell\Documents\Encrypt-Labs\Week3\PhaseC\phase_c.py

The request was denied, producing an AccessDenied error for the s3:ListAllMyBuckets action:

botocore.errorfactory.AccessDenied:

An error occurred (AccessDenied) when calling the ListBuckets operation:

User: arn:aws:iam::[REDACTED]:user/dev-user is not authorized to perform:

s3:ListAllMyBuckets

because no identity-based policy allows the s3:ListAllMyBuckets action

These tests demonstrated that AWS IAM authorization is enforced consistently when requests are made through both the AWS CLI and the Boto3 SDK. The denied responses were captured as evidence of the least-privilege configuration.

Key Observations

- The dev-user was restricted from unauthorized EC2 and S3 operations.
- The ops-user was permitted to perform the tested EC2 and S3 operations due to its higher level of access.

- The results demonstrated the practical effect of IAM permissions and the principle of least privilege.
- Boto3 requests were subject to the same IAM authorization controls as AWS CLI requests.

Evidence Produced

- **Bot_dev-user.png** — Screenshot showing the Boto3-based test performed using the dev-user AWS profile.
- **phase_c.py** — Python/Boto3 script used to programmatically test AWS service access with the dev-user profile.
- **PhaseC-EC2-Dev-user.png** — Screenshot documenting the EC2 permission test performed using the restricted dev-user profile.
- **PhaseC-EC2-Ops-user-Permitted.png** — Screenshot demonstrating that the ops-user profile was permitted to perform the EC2 operation.
- **PhaseC-S3-Denial-Dev-User.png** — Screenshot showing the denied S3 operation when using the restricted dev-user profile.
- **PhaseC-S3-ops-user.png** — Screenshot demonstrating successful S3 access using the ops-user profile.

Phase D: Implement temporary session tokens using MFA for enhanced security.

Phase Objective	Implement temporary session tokens using MFA for enhanced security.
Estimated Time	120 minutes
Evidence File(s)	phaseD_mfa_token.png

Actions Taken

In Phase D, I implemented multi-factor authentication (MFA) and used AWS Security Token Service (STS) to generate temporary session credentials. The purpose was to demonstrate how MFA and short-lived credentials can reduce the security risks associated with compromised long-term access keys.

First, I enabled MFA on the authorized AWS account/identity using the AWS Management Console. MFA provides an additional authentication factor beyond the standard credentials.

I then generated temporary session credentials using the AWS STS command:

aws sts get-session-token --serial-number arn-of-mfa-device --token-code <MFA_CODE> --profile ops-user

The command returned temporary security credentials containing an access key ID, secret access key, session token, and an expiration timestamp.

The temporary credentials were then configured in a separate AWS CLI profile named ops-temp. This profile was used to access AWS services using the temporary session credentials rather than the long-term credentials associated with ops-user.

Finally, I validated the temporary credentials using:

aws sts get-caller-identity --profile ops-temp

The command successfully confirmed the identity associated with the temporary session and demonstrated that the temporary credentials were active. The expiration information was also documented to show that the credentials were time-limited.

Key Observations

- MFA provides an additional layer of protection against unauthorized access.
- Temporary credentials reduce the risk associated with long-term static access keys because they automatically expire.
- The ops-temp profile allowed temporary session credentials to be used separately from the permanent ops-user credentials.
- The STS response demonstrated that temporary credentials have a defined expiration time.
- Combining MFA with temporary credentials provides stronger protection against credential theft and unauthorized AWS access.

Evidence Produced

- **phaseD_mfa_token.png** — Screenshot showing the temporary session token output, including the credential expiration time.

Phase E: Use the labs to reinforce learning

Phase Objective	Use the labs to reinforce learning
Estimated Time	180 minutes
Evidence File(s)	PhaseE_Cyber-Lab-1.png, PhaseE_Cybr_AWS_CLI_Tips.png

Actions Taken

In Phase E, I completed additional hands-on cloud security learning activities to reinforce the concepts and skills developed during the previous phases. I completed the Cybr – Getting Started with the AWS CLI lab, which reinforced AWS CLI configuration, authentication, and IAM-related concepts covered in Phase B.

I also completed **the Cybr – AWS CLI Tips and Tricks lab** to strengthen my understanding of AWS CLI usage and automation concepts relevant to Phases C and D. These practical exercises allowed me to apply AWS CLI commands in a controlled training environment and reinforce my understanding of AWS authentication and cloud security.

Key Observations

- Hands-on labs reinforced the AWS CLI and IAM concepts practiced in the main task.
- Practical exercises improved familiarity with AWS CLI authentication and command usage.
- The labs demonstrated how AWS CLI skills can support cloud security administration and automation.
- Completing activities in authorized training environments provided additional practical experience without accessing production or third-party systems.

Evidence Produced

- PhaseE_Cyber-Lab-1.png— Completion certificate for the Cybr – Getting Started with the AWS CLI lab.

- PhaseE_Cybr_AWS_CLI_Tips.png — Completion for the Cybr – AWS CLI Tips and Tricks lab.

Phase F: Compile all findings into a professional report aligned with ISO 27001 control requirements.

Phase Objective	Compile all findings into a professional report aligned with ISO 27001 control requirements.
Estimated Time	150 minutes
Evidence File(s)	Final_AWS_CLI_Security_Report.pdf

Actions Taken

In Phase F, I consolidated the findings and evidence collected throughout Phases A–E into a professional cloud security report. I reviewed the results of the IAM configuration, AWS CLI authentication, permission testing, Boto3 automation, MFA implementation, temporary credentials, and hosted lab activities.

I analyzed the security risks identified during the practical exercises, including credential leakage, excessive permissions, unauthorized access, and potential privilege escalation. I also reviewed the use of security controls such as least-privilege IAM policies, MFA, temporary credentials, and secure credential management.

The findings were organized and aligned with relevant ISO/IEC 27001 security principles related to access control, identity management, authentication information, and protection of information systems. The report also included supporting screenshots, command outputs, lab completion evidence, and recommendations for improving AWS cloud security.

Key Observations

- IAM least-privilege controls reduce the potential impact of compromised credentials.
- MFA provides an additional layer of protection against unauthorized account access.
- Temporary credentials reduce the risk associated with long-term static access keys.
- Poor credential management and excessive permissions remain significant cloud security risks.
- Regular IAM permission reviews and secure credential management are important for maintaining a secure AWS environment.

- The practical findings can be mapped to broader information security and access-control requirements within ISO/IEC 27001.

Evidence Produced

- Final_AWS_CLI_Security_Report.pdf — Final professional report consolidating the findings, methodology, evidence, threat analysis, and security recommendations from Phases A–E.
- Phase A evidence — AWS CLI identity verification and IAM credential analysis.
- Phase B evidence — IAM user and AWS CLI profile configuration evidence.
- Phase C evidence — EC2 and S3 permission testing, including permitted and denied access results and Boto3 testing.
- Phase D evidence — MFA and temporary credential evidence, including token expiration output.
- Phase E evidence — Completion from the required hosted AWS CLI training labs.

4.0 Findings

4.1 Findings Summary

#	Finding Title	Type / Category	Cloud Service / Resource	MITRE Ref.	Phase	Severity	Evidence Ref.
1	Excessive Privileges Assigned to Administrative IAM User	Access Control / IAM Policy Gap	AWS IAM ops-user	T1078 – Valid Accounts	Phase B/C	Medium	PhaseC-EC2-Ops-user-Permitted.png, PhaseC-S3-ops-user.png
2	Least-Privilege Controls Successfully Prevented Unauthorized Access	Access Control / Security Control Validation	AWS IAM dev-user, EC2, S3	T1078 – Valid Accounts	Phase C	Low	PhaseC-EC2-Dev-user.png, PhaseC-S3-Denial-Dev-User.png
3	Long-Lived IAM Access Keys Present Credential Exposure Risk	Credential Management / Authentication Weakness	AWS IAM Users / AWS CLI Profiles	T1078 – Valid Accounts	Phase B/D	High	AWS CLI profile configuration evidence; MFA token evidence
4	MFA and Temporary Credentials Reduce Credential Theft Risk	Authentication / Security Control	AWS STS / MFA / ops-temp	T1078 – Valid Accounts	Phase D	Low	phaseD_mfa_token.png
5	IAM Permissions Require Regular Review	Access Control / Privilege Management	AWS IAM dev-user and ops-user	T1087.004 – Cloud Account Discovery	Phase B/C	Medium	Bot_dev-user.png, PhaseC-EC2-Dev-user.png, PhaseC-EC2-Ops-user-Permitted.png

4.2 Detailed Finding Analysis

Finding 1: Excessive Privileges Assigned to Administrative IAM User

Field	Details
Finding Title	*Excessive Privileges Assigned to Administrative IAM User*
Type / Category	*Access Control Weakness / IAM Policy Gap / Privilege Management*
Cloud Service / Resource	*AWS IAM ops-user; AWS EC2 and Amazon S3*
Description	*The ops-user account was configured with administrative-level permissions. During testing, the account was able to successfully perform the EC2 and S3 operations used in the lab. While this behavior was expected for the administrative test account, broad privileges increase the potential impact if the credentials are compromised. Administrative accounts should be restricted to users who require elevated permissions and should be protected with strong authentication controls.*
How Identified	*The finding was identified by configuring the ops-user AWS CLI profile and testing AWS services using commands such as aws ec2 describe-instances --profile ops-user and aws s3 ls --profile ops-user. The successful operations demonstrated that the account had broader permissions than the restricted dev-user.*
MITRE ATT&CK Reference	*T1078 – Valid Accounts. An attacker who obtains valid administrative AWS credentials could use the account to access cloud resources and perform authorized actions.*
Business / Security Impact	*If administrative credentials are compromised, an attacker could potentially access or modify AWS resources, change IAM permissions, or perform other privileged operations. This could affect confidentiality, integrity, and availability of cloud resources.*
Severity / Impact	*Medium*
Evidence Reference	*PhaseC-EC2-Ops-user-Permitted.png and PhaseC-S3-ops-user.png*

Finding 2: Long-Term IAM Access Keys Present Credential Exposure Risk

Field	Details
Finding Title	*Long-Term IAM Access Keys Present Credential Exposure Risk*
Type / Category	*Credential Management / Authentication Weakness*
Cloud Service / Resource	*AWS IAM users dev-user and ops-user; AWS CLI credential profiles*
Description	*IAM access keys were generated and configured for AWS CLI authentication. These static credentials provide ongoing access until they are disabled, deleted, rotated, or otherwise invalidated. If an access key and its associated secret key are accidentally exposed through insecure storage, source code, logs, or other channels, an unauthorized party could potentially authenticate to AWS using the affected identity.*
How Identified	*The finding was identified during AWS CLI profile configuration and validation. The profiles were configured with access keys and secret access keys and subsequently used to authenticate AWS CLI commands. The active identity was verified using AWS STS.*

MITRE ATT&CK Reference	*T1078 – Valid Accounts. Compromised AWS access keys could allow an attacker to authenticate using a valid cloud identity.*
Business / Security Impact	*Compromised access keys could result in unauthorized access to AWS resources. The potential impact depends on the permissions assigned to the associated IAM user. Administrative credentials would present a greater risk than restricted credentials.*
Severity / Impact	*Medium*
Evidence Reference	*AWS CLI profile configuration evidence and identity verification output from aws sts get-caller-identity.*

Finding 3: MFA and Temporary Credentials Reduce Credential Theft Risk

Field	*Details*
Finding Title	*MFA and Temporary Credentials Reduce Credential Theft Risk*
Type / Category	*Security Control / Authentication Improvement*
Cloud Service / Resource	*AWS IAM / AWS STS / MFA / ops-temp profile*
Description	*MFA was enabled and temporary session credentials were generated for authenticated AWS access. The temporary credentials were configured in the ops-temp profile and validated using AWS STS. Unlike long-term access keys, temporary credentials have a defined expiration time, reducing the period during which compromised credentials can be used. MFA provides an additional authentication factor that helps reduce the risk of unauthorized access when a password or other credential is compromised.*
How Identified	*The control was validated by generating temporary session credentials using AWS STS and configuring the resulting credentials for the ops-temp profile. The temporary identity was then tested using aws sts get-caller-identity --profile ops-temp. The token expiration output was captured as phaseD_mfa_token.png.*
MITRE ATT&CK Reference	*T1078 – Valid Accounts. MFA and temporary credentials are defensive controls that reduce the likelihood and duration of successful abuse of valid cloud credentials.*
Business / Security Impact	*MFA strengthens authentication by requiring an additional factor. Temporary credentials reduce the exposure window because they automatically expire. Together, these controls reduce the likelihood and potential duration of unauthorized access resulting from credential theft.*
Severity / Impact	*Informational / Positive Security Control*
Evidence Reference	*phaseD_mfa_token.png*

5.0 Risk Assessment & Cloud Threat Model

Not Applicable: The security monitoring and detection services listed in this subsection were not used during the practical phases of this task. As a result, no telemetry or evidence was collected from these tools. To maintain the accuracy and integrity of the report, no findings or controls from these services have been claimed as implemented or tested. The assessment is limited to the AWS services and tools actually used during the lab exercises.

5.1 Cloud Security Control Gaps & Defensive Coverage

[For each significant finding, describe which AWS-native or third-party control could detect, prevent, or reduce the risk. Consider: CloudTrail rules, AWS Config rules, GuardDuty detectors, IAM Access Analyzer, Security Hub checks, S3 Block Public Access, SCPs, or CloudWatch alarms.]

Finding / Gap	Observable / Trigger in Cloud Telemetry	Recommended Control	AWS Service / Tool for Implementation
[Finding 1]	*[e.g. GetObject requests from unexpected IP in CloudTrail logs]*	*[e.g. Enable S3 Block Public Access + bucket policy denying public GetObject]*	*[e.g. AWS S3 Console / AWS Config rule: s3-bucket-public-read-prohibited]*
[Finding 2]	*[e.g. AttachUserPolicy event by non-admin user in CloudTrail]*	*[e.g. SCPs restricting IAM write actions; IAM Access Analyzer alert rule]*	*[e.g. AWS Organizations SCP / IAM Access Analyzer / CloudWatch Alarm]*
[Finding 3]			

5.2 Risk Exposure Over Time

[For the top 2–3 risks identified in this task, explain what happens if they remain unaddressed over time. Consider: persistence, lateral movement, cross-account escalation, data exfiltration, regulatory non-compliance, or loss of audit visibility. Write 2–4 sentences per risk. Ground every statement in your actual task findings — not generic cloud security advice.]

Risk 1 — [Title]:
[Your analysis — tied to specific findings from Section 4]

Risk 2 — [Title]:
[Your analysis — tied to specific findings from Section 4]

Risk 3 — [Title — remove if fewer than 3 top risks]:
[Your analysis]

6.0 Recommendations

6.1 Technical Remediation

#	Finding	Specific Remediation Action	Priority	Standard / Reference
1	Excessive Privileges Assigned to Administrative IAM User	Review the permissions assigned to ops-user and remove permissions that are not required for its operational duties. Apply least privilege and restrict administrative access to authorized users. Require MFA for privileged access.	Short-term	ISO/IEC 27001:2022 access control principles; CIS AWS Foundations Benchmark
2	Long-Term IAM Access Keys Present Credential Exposure Risk	Replace long-term access keys with temporary credentials where possible. Rotate or disable unused access keys and avoid storing credentials in source code or insecure locations. Use AWS STS temporary credentials and MFA for privileged access.	Immediate	ISO/IEC 27001:2022 authentication information and access control principles; CIS AWS Foundations Benchmark
3	MFA and Temporary Credentials Reduce Credential Theft Risk	Continue enforcing MFA for privileged accounts and use short-lived STS credentials. Regularly review authentication and credential configurations to ensure temporary credentials are used where appropriate.	Short-term	ISO/IEC 27001:2022 access control and secure authentication principles; CIS AWS Foundations Benchmark

6.2 Strategic & Operational Improvements

- **Establish a least-privilege IAM governance process:** Regularly review IAM users, roles, and attached policies to ensure that permissions are limited to the minimum required for each user's responsibilities. Administrative privileges should be restricted to authorized personnel and reviewed periodically.
- **Adopt temporary credentials as the preferred authentication method:** Reduce organizational reliance on long-term IAM access keys by using AWS STS temporary credentials and role-based access wherever possible. This limits the useful lifetime of compromised credentials and reduces the impact of credential leakage.
- **Enforce MFA for privileged accounts:** Require multi-factor authentication for administrative and other high-privilege IAM accounts. MFA should be incorporated into the organization's access-control policy and verified during periodic security reviews.
- **Implement regular credential and access reviews:** Establish a scheduled process to review IAM users, access keys, permissions, and account activity. Unused credentials should be disabled or removed, while access should be revoked promptly when users no longer require it.
- **Introduce security testing into operational procedures:** Include IAM permission testing and access-control validation as part of routine cloud security assessments. Both authorized and unauthorized access scenarios should be tested to confirm that least-privilege policies operate as intended and that users cannot perform actions outside their assigned responsibilities.

7.0 Reflection

Q1 How does enforcing least privilege reduce business risk in multi-account environments?

Enforcing least privilege limits each user or role to only the permissions required for their responsibilities. Based on my lab, the dev-user could perform permitted read-only activities but was denied unauthorized administrative actions. This reduces the potential impact of compromised credentials, accidental changes, and privilege escalation across cloud accounts.

Q2 What are potential consequences if an intern accidentally commits .aws/credentials to GitHub?

If AWS credentials are committed to a public or unauthorized repository, an attacker could obtain the access keys and use them to access AWS resources. Depending on the permissions assigned to the credentials, this could result in unauthorized data access, resource modification, financial costs from resource abuse, or privilege escalation. Credentials should be revoked immediately, and replacement credentials should be generated.

Q3 How can MFA integration improve compliance with ISO 27001 controls?

MFA strengthens access security by requiring users to provide an additional verification factor beyond a password or access key. This helps reduce the risk of unauthorized access if credentials are stolen or compromised and supports ISO 27001 objectives related to secure authentication and access control.

Q4 In what ways could automation scripts unintentionally expose secrets?

Automation scripts can expose secrets by hardcoding access keys in source code, storing credentials in plain-text configuration files, printing secrets in logs, or including credentials in error messages. Secrets can also accidentally be committed to version-control systems such as GitHub. Using temporary credentials, environment variables, and secure secrets management can reduce these risks.

Q5 How would you communicate a detected misconfiguration risk to management?

I would explain the issue in clear business terms by describing what was misconfigured, what resource was affected, the likelihood of exploitation, and the potential business impact. I would assign a severity and priority, provide evidence supporting the finding, and recommend specific remediation steps. For example, I would explain that excessive IAM permissions could increase the impact of a compromised account and recommend applying least privilege and MFA to reduce the risk.

8.0 Deliverables Submission Checklist

✓	Phase	Deliverable Description	Your Filename	Status
☐	Phase A	Theory evidence — IAM credentials, user-based vs. role-based authentication, identity verification, and static vs. temporary credentials	PhaseA_identity.png, Phase_A_AWS_IAM_Credentials_and_Identity_Context.pdf	Complete
☐	Phase B	Environment setup evidence — IAM users, access keys, AWS CLI profiles, and get-caller-identity output	PhaseB-Dev-User.png, PhaseB-ops-user.png	Complete
☐	Phase C	Core execution evidence — IAM permission testing using dev-user and ops-user, EC2/S3 access results, and Boto3 testing	Bot_dev-user.png, phase_c.py, PhaseC-EC2-Dev-user.png, PhaseC-EC2-Ops-user-Permitted.png, PhaseC-S3-Denial-Dev-User.png, PhaseC-S3-ops-user.png	Complete
☐	Phase D	MFA and temporary credential evidence — MFA configuration, STS session token generation, temporary profile, and token expiration verification	phaseD_mfa_token.png	Complete
☐	Phase E	Hosted lab integration — completion certificates from the required Cybr/PwnedLabs labs	PhaseE_Cyber-Lab-1.png, PhaseE_Cybr_AWS_CLI_Tips.png	Complete
☐	Phase F	Final professional report containing findings, risk assessment, threat model, recommendations, reflection, and supporting evidence	Final_AWS_CLI_Security_Report.pdf	Complete

Submission naming convention:

- Report PDF: CSE_CSE-AWS-L-003-04 _S.K_19940225.pdf
- Evidence folder: CSE-AWS-L-003-04 _Evidence_S.K — zip all phase files together

9.0 Additional Observations

No additional observations for this task

Declaration & Sign-Off

I confirm that:

- All work in this report is my own except where AI usage is explicitly declared in Section 2.3.
- All technical activities were performed exclusively within the authorised cloud environments listed in Section 1.3.
- No external AWS accounts, production environments, third-party cloud resources, or real customer data were accessed during this task.
- All AWS credentials, access keys, session tokens, and account identifiers used during this task have been redacted in evidence files before submission.
- I have reviewed this report for accuracy, completeness, and professional presentation before submission.

Intern Full Name	Signature	Date of Submission
Sukhmandeep Kaur		*22 July 2026*

EncryptEdge Labs Limited — Junior Cloud Security Engineer Internship Programme
Company No. 15830711 | United Kingdom | encryptedgelabs.com